

Build Strategy Options

How the new code actually gets onto the EC2 instance during a deploy · prepared 2026-06-21

Once the deploy step can *reach* the instance, it must get the new version of the app onto the box and run it. There are two common strategies. This document explains both; the **git pull + build on EC2** strategy is implemented first because it is the simplest and matches what your Terraform `user_data` already does.

At a glance

	Option 1 — git pull + build on EC2	Option 2 — push image to registry
Where the image is built	On the EC2 instance	On the GitHub runner (then pushed)
Needs a registry?	No	Yes (Docker Hub or AWS ECR)
EC2 does	<code>git pull</code> + <code>docker build</code> + <code>docker run</code>	<code>docker pull</code> + <code>docker run</code>
Build load on EC2	Yes (t3.micro builds each deploy)	No (just pulls a ready image)
Rollback by tag	Harder (rebuild old commit)	Easy (run a previous image tag)
Difficulty	Lowest — implement this first	Moderate

Option 1 — git pull + build on EC2 implemented first

The deploy step tells the instance to pull the latest commit and build the Docker image locally, then restart the container. This mirrors your existing Terraform `user_data`, so there is nothing new to learn about registries.

```
on EC2:
cd /opt/app
git pull          # fetch new code
docker build -t my-devops-app:latest .
docker rm -f my-devops-app # stop old container
docker run -d --restart=always -p 3000:3000 my-devops-app:latest
```

Pros: simplest; no registry or extra credentials; consistent with current setup; one place builds and runs.

Cons: the small t3.micro rebuilds on every deploy (slower, more CPU/RAM); rollbacks mean rebuilding an older commit; build tools live on the production box.

Steps to implement

1. Ensure the repo is cloned on the box at `/opt/app` (your `user_data` already does this on first boot).
2. In the deploy job, run the five commands above on the instance (over SSH or SSM).
3. Add a `docker image prune -f` at the end to stop old images filling the small disk.
4. Health-check after restart: `curl -fsS http://localhost:3000/health`.

Option 2 — push image to registry production-like

The GitHub runner builds the image once (it already builds it for the Trivy scan), tags it with the commit SHA, and pushes it to a registry. The EC2 instance only *pulls* the finished image and runs it — no building on prod.

```
GitHub runner:
  docker build -t /my-devops-app: .
  docker push /my-devops-app:
EC2:
  docker pull /my-devops-app:
  docker run -d --restart=always -p 3000:3000 /my-devops-app:
```

Pros: EC2 never builds (faster deploys, less load on t3.micro); immutable, SHA-tagged images; trivial rollback (run an older tag); the exact scanned image is what runs.

Cons: needs a registry (Docker Hub free, or AWS ECR) and registry credentials in GitHub (or an IAM role for ECR).

Steps to implement (ECR variant)

1. **Create an ECR repository** (console, CLI, or Terraform `aws_ecr_repository`).
2. **Give GitHub permission to push** — AWS credentials/OIDC, then `aws-actions/amazon-ecr-login` .
3. **Extend the build job** to tag & `push` the image to ECR (reuse `docker/build-push-action` with `push: true`).
4. **Give EC2 pull permission** — an IAM role with `AmazonEC2ContainerRegistryReadOnly` .
5. **Deploy job** tells the instance to `aws ecr get-login-password | docker login` , `docker pull` the new SHA tag, and restart the container.

```
# build job addition (Docker Hub variant)
- uses: docker/login-action@v3
  with: { username: ${ secrets.DOCKERHUB_USER }, password: ${ secrets.DOCKERHUB_TOKEN } }
- uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: youruser/my-devops-app:${ github.sha }
```

Recommendation

Start with **Option 1 (git pull + build on EC2)** — it is the fewest moving parts and matches your current Terraform. Once the full pipeline works end-to-end, switch to **Option 2 (registry)** to get faster deploys, immutable images, and one-command rollbacks.